

SKILL EXPERIMENT – 16

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

Implementation of Background Job Detection and Job Table Management

Aim

To detect background jobs, launch non-blocking processes, return the command prompt immediately, store job information, monitor job execution, and test background tasks.

To create a job table, store process groups, track job states, update job status, remove completed jobs, and validate job records.

Theory

Background job management allows processes to execute without blocking the command prompt. A process launched as a background job can continue execution while the shell remains available to accept new commands.

The shell maintains information about active jobs using a job table. Each job record can contain a job ID, process ID or process group ID, command information, and the current execution state. The job state is updated as the process starts, continues, stops, or completes.

Completed jobs are removed from the job table after their termination status is detected. Maintaining and validating job records helps the shell correctly monitor active and completed background processes.

Algorithm

1. Start the program.
2. Initialize the job table.
3. Read a command or background task from the user.
4. Detect whether the task is requested as a background job.
5. Create a new process using the appropriate process-creation mechanism.
6. Launch the background process without blocking the parent process.
7. Return the prompt immediately to the user.

8. Assign a unique job ID to the background process.
9. Store the job ID, process ID, process group information, command, and state in the job table.
10. Continue accepting new commands while the background process executes.
11. Monitor the background process for state changes.
12. Update the job state when the process changes its execution status.
13. Detect when a background process has completed.
14. Remove the completed job from the job table.
15. Validate the remaining job records.
16. Display the current background jobs and their states.
17. Repeat the process until the user exits.
18. Stop the program.

Code

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>


typedef struct {

    int job_id;

    pid_t pid;

    char state[20];

} Job;


int main() {

    Job jobs[3];

    int count = 0;


    printf("Starting background jobs...\n\n");


    for (int i = 0; i < 3; i++) {

        pid_t pid = fork();
```

```
if (pid == -1) {  
  
    perror("fork");  
  
    return 1;  
  
}
```

```
if (pid == 0) {  
  
    printf("Job running: PID = %d\n", getpid());  
  
    sleep(3);  
  
    printf("Job completed: PID = %d\n", getpid());  
  
    exit(0);  
  
}
```

```
jobs[count].job_id = count + 1;  
  
jobs[count].pid = pid;  
  
sprintf(jobs[count].state, "Running");  
  
count++;
```

```
printf("[Job %d] Started | PID = %d | State = Running\n",  
  
    count, pid);  
  
}
```

```
printf("\nJob Table:\n");  
  
for (int i = 0; i < count; i++) {  
  
    printf("Job %d | PID = %d | State = %s\n",  
  
        jobs[i].job_id,  
  
        jobs[i].pid,  
  
        jobs[i].state);  
  
}
```

```
printf("\nWaiting for background jobs...\n");

for (int i = 0; i < count; i++) {

    waitpid(jobs[i].pid, NULL, 0);

    sprintf(jobs[i].state, "Completed");

}

printf("\nUpdated Job Table:\n");

for (int i = 0; i < count; i++) {

    printf("Job %d | PID = %d | State = %s\n",

        jobs[i].job_id,

        jobs[i].pid,

        jobs[i].state);

}

printf("\nAll background jobs completed successfully.\n");

return 0;

}
```

Compilation

```
hp@DESKTOP-35T9S8P:~$ nano expl.c
hp@DESKTOP-35T9S8P:~$ gcc expl.c -o expl
hp@DESKTOP-35T9S8P:~$ ./expl
Starting background jobs...

[Job 1] Started | PID = 514 | State = Running
Job running: PID = 514
[Job 2] Started | PID = 515 | State = Running
Job running: PID = 515
[Job 3] Started | PID = 516 | State = Running

Job Table:
Job 1 | PID = 514 | State = Running
Job 2 | PID = 515 | State = Running
Job 3 | PID = 516 | State = Running

Waiting for background jobs...
Job running: PID = 516
Job completed: PID = 514
Job completed: PID = 515
Job completed: PID = 516

Updated Job Table:
Job 1 | PID = 514 | State = Completed
Job 2 | PID = 515 | State = Completed
Job 3 | PID = 516 | State = Completed

All background jobs completed successfully.
```

Result

The background job management system was implemented successfully. Background processes were launched without blocking the prompt, job information was stored in a job table, process states were monitored and updated, and completed jobs were removed after execution.